

Лабораторная 6. Линейный, бинарный и интерполяционный поиск

Контракт поиска

Функция возвращает пару: индекс найденного элемента и количество сравнений ключей. Если ключ отсутствует, индекс равен -1 . При повторях бинарный и интерполяционный поиск могут вернуть любое подходящее вхождение; линейный возвращает первое. Это не ошибка: контракт не требует самого левого индекса. Требования к распределениям, счётчикам и граничным случаям заданы в КИМ-06.[1]

Для бинарного и интерполяционного поиска массив должен быть отсортирован по неубыванию. Проверка сортировки не встроена в функцию: она заняла бы $O(n)$ и скрыла преимущество быстрого поиска. Предусловие обеспечивается вызывающим кодом и тестами.

Линейный поиск

Проходим по элементам от начала и сравниваем каждый с ключом. Увидели совпадение — возвращаем индекс. Дошли до конца — возвращаем -1 . Инвариант: все уже просмотренные элементы не равны искомому ключу, иначе функция уже закончилась бы.

Успех в первом элементе стоит одно сравнение: лучший случай $\Theta(1)$. Отсутствующий ключ требует n сравнений: худший $\Theta(n)$. Если успешная позиция равновероятна среди n различных элементов, среднее число сравнений $(n+1)/2$. Память $O(1)$, предварительная сортировка не нужна.

Бинарный поиск: отбрасываем половину

Границы lo и hi включаются в рассматриваемый участок $[lo, hi]$. Пока $lo \leq hi$, вычисляется $mid = (lo + hi) // 2$. Если $a[mid]$ равен ключу, поиск закончен. Если $a[mid]$ меньше, ни он, ни элементы левее не подходят, поэтому $lo = mid + 1$. Иначе $hi = mid - 1$.

Пример: массив $[2, 5, 8, 12, 16, 23, 38]$, ключ 23. Сначала $lo=0$, $hi=6$, $mid=3$, значение 12: ищем правее. Затем $lo=4$, $hi=6$, $mid=5$, значение 23: найден индекс 5. Для ключа 15 второй шаг сузил бы границы к 16, затем участок стал бы пустым.

Инвариант: если нужное значение существует и ещё не найдено, хотя бы одно подходящее вхождение находится внутри текущего участка. Переходы с $+1$ и -1 гарантируют уменьшение участка. Ошибка $lo = mid$ при двух элементах может оставить границу на месте и вызвать бесконечный цикл.

После k неуспешных шагов длина участка не больше $n/2^k$. При k порядка $\log_2 n$ элементов не остаётся. Худшее время $O(\log n)$, лучшее $\Theta(1)$, память $O(1)$. В нашей функции проверка равенства и проверка порядка считаются двумя отдельными сравнениями, поэтому $comparisons$ не равно числу итераций.

Интерполяционный поиск: оцениваем позицию по значению

Если числовые ключи распределены почти равномерно, можно угадать позицию. Пусть $a[lo]=10$, $a[hi]=110$, а искомое значение 30. Оно находится примерно на одной пятой числового диапазона, поэтому проверяем около одной пятой индексного диапазона.

Формула: $pos = lo + [(target - a[lo]) \cdot (hi - lo) / (a[hi] - a[lo])]$. После сравнения границы меняются так же, как в бинарном поиске, но следующая позиция снова оценивается по значениям, а не просто берётся середина.

Сначала проверяется, входит ли `target` в диапазон крайних значений. Если нет, он не может присутствовать в отсортированном участке. Если края равны, делить на их разность нельзя. Но при прошедших проверках диапазона `target` уже равен обоим краям; можно вернуть `lo`. Этот порядок проверок объясняет, почему отдельное равенство `target == a[lo]` здесь не требуется.

При подходящей модели равномерного распределения ожидаемое число проб $O(\log \log n)$: хорошая оценка позиции быстрее сокращает неопределённость. Это не гарантия для каждого массива и не следствие одной только сортировки. Время также зависит от вычисления формулы и числа сравнений границ.

Как построить плохой вход

Возьмём $[0, 1, 2, \dots, n-2, 10^{12}]$ и будем искать $n-2$. Огромный последний элемент растягивает диапазон значений. Формула постоянно оценивает позицию около левого края, поэтому `lo` продвигается очень медленно. Получается $\Theta(n)$ шагов вместо $O(\log n)$.

На экспоненциально распределённых ключах плотность в разных частях диапазона неодинакова; линейная оценка позиции становится менее точной. Однако конкретный конечный набор не обязан всегда давать больший счётчик, чем бинарный поиск. Вывод делается по измерениям, а явный пример с выбросом отдельно демонстрирует худший случай.

Прикладная защита — ограничить число интерполяционных шагов и перейти к бинарному поиску, либо периодически делать бинарный шаг. В текущей работе реализован исходный интерполяционный алгоритм с защитой от деления на ноль и строгим уменьшением границ. Гибридный вариант обсуждается, но не выдаётся за измеренную реализацию.

Когда окупается предварительная сортировка

Пусть нужно q запросов к одному неизменному массиву. Линейный поиск обходится примерно в $O(qn)$. Сортировка и бинарные запросы — $O(n \log n + q \log n)$. Асимптотически предварительная сортировка начинает быть привлекательной при числе запросов порядка $\log n$ или больше, но точный порог зависит от констант, доли успешных запросов и структуры данных.

Для одного запроса сортировать массив обычно невыгодно. Если после каждого запроса данные меняются, стоимость поддержания порядка тоже учитывается. В лабораторном графике измеряется только поиск на уже подготовленном массиве, не полный жизненный цикл задачи.

Воспроизведение опыта

```
python -m pytest labs/lab06 -q
python -m labs.lab06.experiment --variant 15
```

Для каждого из пяти размеров 1000...100000 генератор NumPy с `seed 45` создаёт равномерный и экспоненциальный набор. Значения умножаются на два: все ключи чётные. Успешные запросы выбираются из массива, неуспешные получаются добавлением единицы. Поэтому они гарантированно отсутствуют, но остаются близки к

тому же диапазону, а не все отбрасываются одной проверкой за пределами массива.

Каждый опыт использует 64 запроса; сохраняется среднее число сравнений и медиана времени одного запроса по пяти повторам. Сравняются одинаковые запросы на одинаковом массиве. Подготовка и сортировка выполняются вне таймера. Отдельный набор `worst_case` содержит пример с одним огромным выбросом.

Что проверяют тесты

Пустой массив, один элемент, одинаковые значения, отрицательные числа, начало и конец, отсутствующий ключ, большой разрыв между значениями. На 400 случайных массивах наличие сверяется с `bisect_left`. При найденном индексе дополнительно проверяется `a[index]==target`. Нельзя требовать равенства индексов с `bisect_left` при дубликатах, если контракт допускает любое вхождение.

Вопросы на защите и ответы

1. Какой инвариант нужен бинарному поиску? Неотброшенные подходящие значения находятся внутри включительных границ `[lo,hi]`. Каждый неуспешный шаг строго уменьшает участок, иначе возможен бесконечный цикл.

2. Когда бинарный поиск лучше линейного с учётом сортировки? Когда один отсортированный массив обслуживает достаточно много запросов. Сравняются qn и $n \log n + q \log n$; одиночный запрос обычно не оправдывает сортировку.

3. Откуда идея интерполяции и оценка $O(\log \log n)$? Позиция угадывается по относительному положению ключа в числовом диапазоне. Для подходящих равномерных случайных данных оценка намного точнее простого деления пополам; заявленная сложность — ожидаемая при этих предположениях.

4. Каков худший случай и как защититься? Неравномерный диапазон с большим выбросом способен давать только по одному исключённому элементу, то есть $O(n)$. Гибрид с бинарными шагами ограничивает такую деградацию.

5. Почему не проверяем сортировку внутри функции? Линейная проверка сделала бы каждый вызов как минимум $O(n)$. В учебном интерфейсе порядок является предусловием.

6. Что считается сравнением? Каждая фактически выполненная проверка отношений ключей, включая крайние значения в интерполяции. Сравнения индексов и арифметические операции в счётчик не включаются, но входят во время.

7. Почему меньше сравнений не всегда значит меньше времени? Интерполяция вычисляет умножение и целочисленное деление. Бинарный шаг проще; постоянные расходы могут перевесить экономию нескольких сравнений.

8. Почему все отсутствующие запросы не взяты больше максимума? Тогда интерполяция завершалась бы сразу на проверке диапазона. Нечётные запросы между чётными ключами дают более содержательное сравнение.

Источники

[1] МТУСИ. КИМ-06, 2026. Формулы и примеры в конспекте разобраны применительно к приложенной реализации.

Отчёт по ЛР 6. Поиск в массиве

Цель и выполненная работа

Реализованы алгоритмы и проверки, описанные в конспекте. Эксперимент выполнен на данных варианта 15, seed 45. В таблицах приведены фактические результаты запуска; время указано в секундах, если не оговорено иное.

Условия и методика

CPU: 12th Gen Intel(R) Core(TM) i7-12650H. ОС: Windows-11-10.0.26200-SP0. Python 3.12.12, NumPy 2.5.3, Matplotlib 3.11.1. Дата и время UTC: 2026-09-10T18:29:36.066560+00:00.

Один прогрев и пять повторов на точку, итог — медиана `time.perf_counter`. Ввод, генерация и подготовка свежего состояния исключены из таймера. Фоновые процессы и питание не контролировались; задержки рабочего компьютера могут влиять на отдельные точки. Контрольные суммы данных находятся в `environment.json`.

Результаты и их объяснение

Распределение	Запрос	Алгоритм	Сравнений/запрос	с/запрос
uniform	hit	linear_search	48479.1	0.00398298
uniform	hit	binary_search	30.375	2.90156e-06
uniform	hit	interpolation_search	20.1719	1.76406e-06
uniform	miss	linear_search	100000	0.00829491
uniform	miss	binary_search	33.4062	2.91406e-06
uniform	miss	interpolation_search	21.7344	1.68906e-06
exponential	hit	linear_search	51108.7	0.00448951
exponential	hit	binary_search	26	2.22969e-06
exponential	hit	interpolation_search	512.906	3.8825e-05
exponential	miss	linear_search	100000	0.00844681
exponential	miss	binary_search	33.3438	4.43594e-06
exponential	miss	interpolation_search	527.719	4.42422e-05

Сравнение учитывает проверки крайних значений у интерполяции. На каждом массиве 64 успешных и 64 отсутствующих запроса; отсутствие гарантировано чётностью ключей. Равномерная модель помогает оценке позиции, но меньшее число проб не обязательно компенсирует стоимость арифметики и дополнительных сравнений.

Алгоритм	n	Сравнения	c
binary_search	10000	25	5.20001e-06
interpolation_search	10000	49994	0.0059032

Контрольный вход $[0, 1, \dots, n-2, 10^{12}]$ показывает линейную деградацию интерполяции при поиске $n-2$. Основные экспоненциальные наборы и конструктивный худший случай рассматриваются отдельно: само название распределения ещё не доказывает

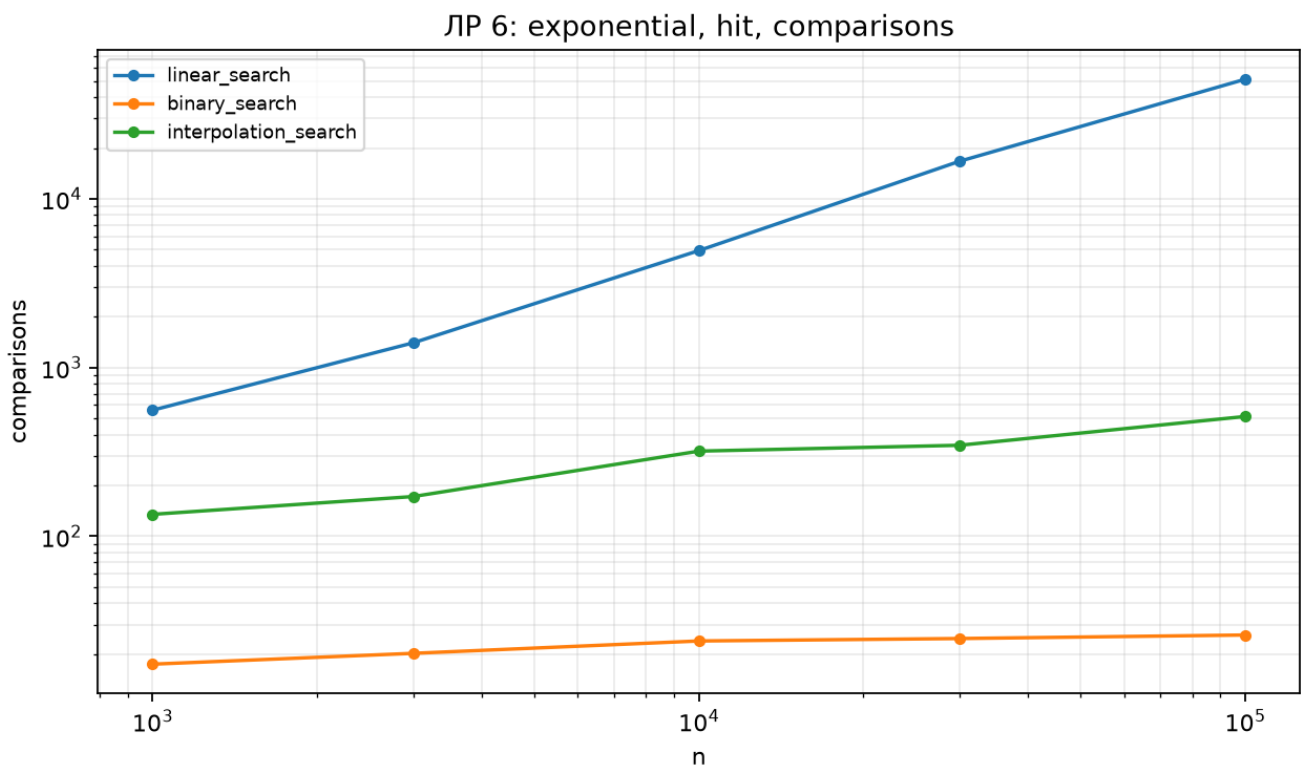
конкретный результат.

Проверка и вывод о применимости

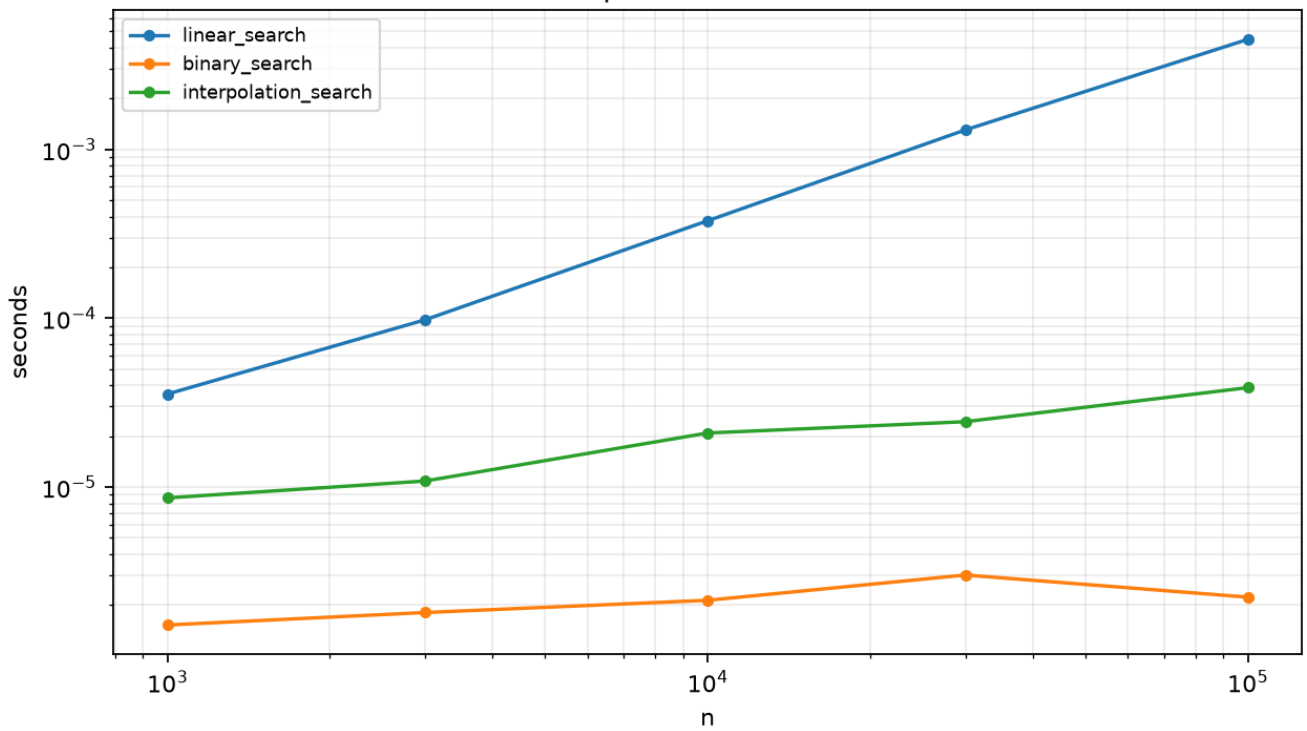
Проверки находятся в `test_lab.py`. Запуск: `python -m pytest labs/lab06 -q`. Применены граничные случаи, инварианты и независимые эталоны; состав тестов подробно разобран в конспекте. Проверки всего комплекта также сохранены в `verification.txt` в корне репозитория.

Вывод: принять для учебного использования в рамках явно описанных контрактов. Измерения подтверждают основные тенденции роста; ограничения реализаций и сравнения указаны выше и в конспекте. Автоматическая проверка не оценивает готовность к устной защите.

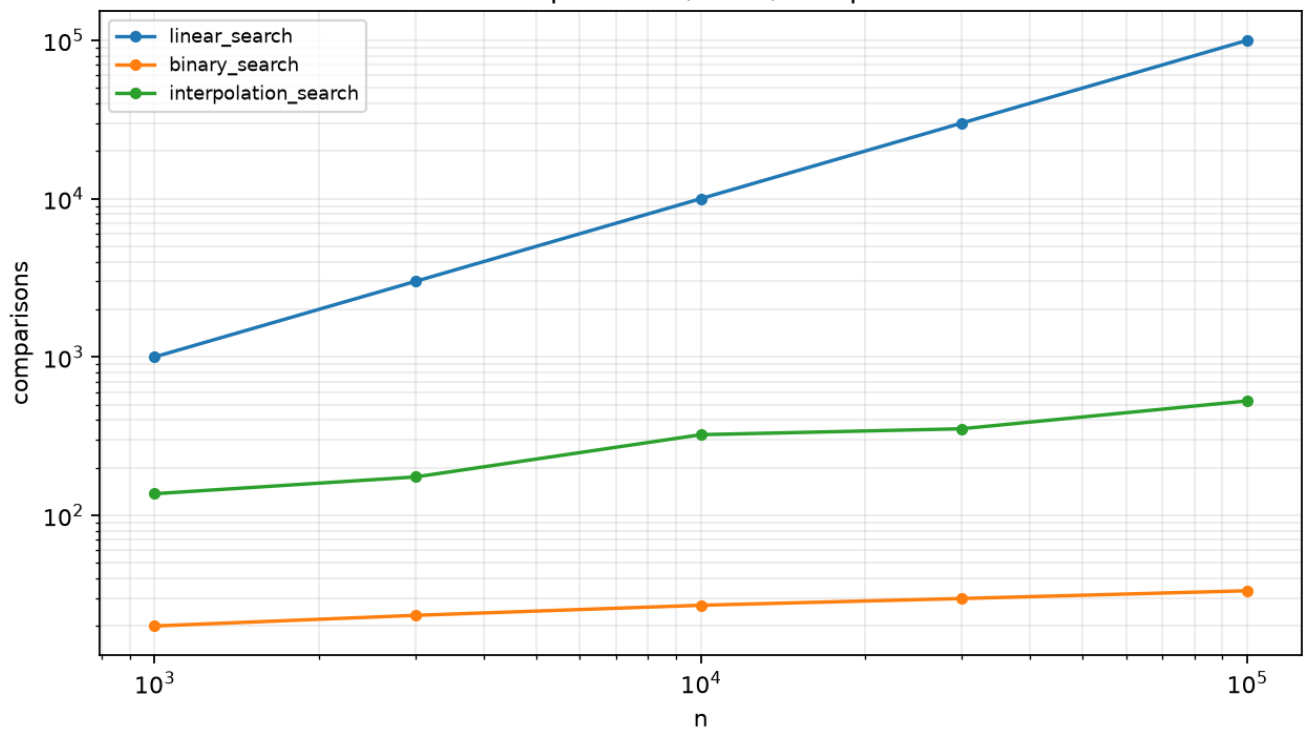
Графики



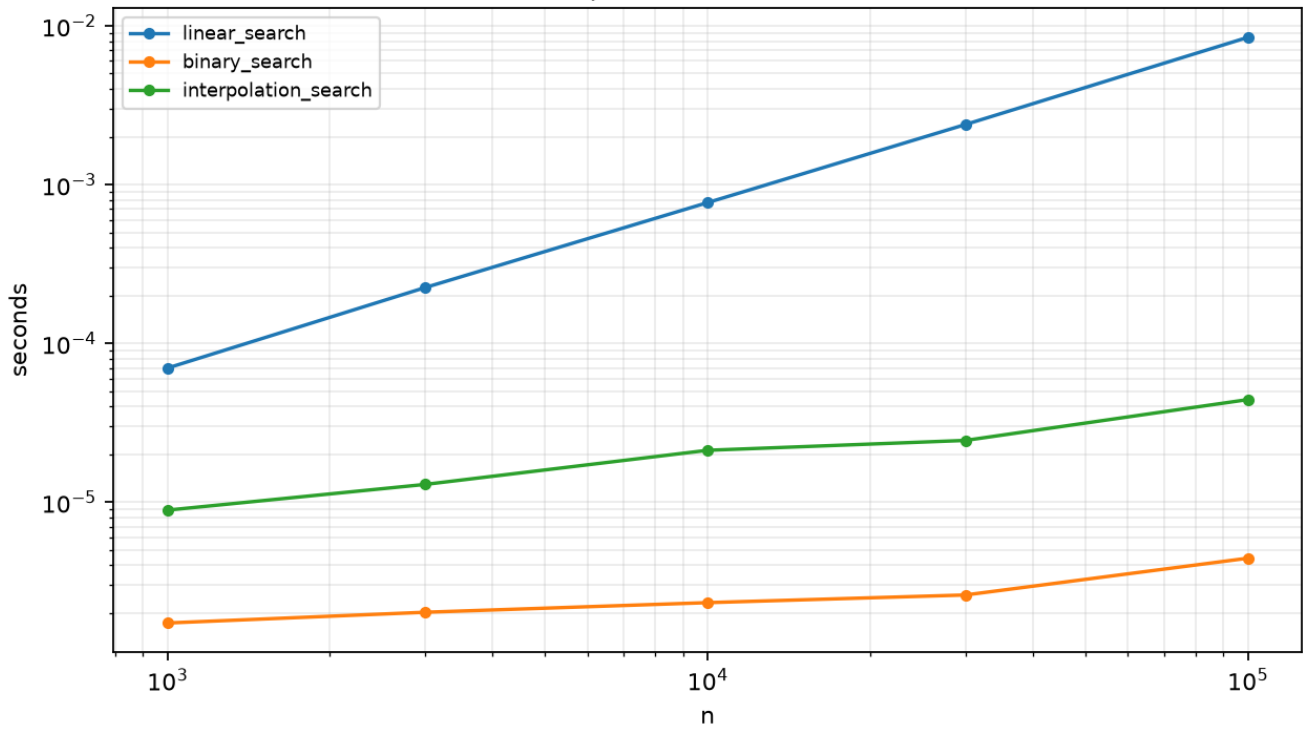
ЛР 6: exponential, hit, seconds



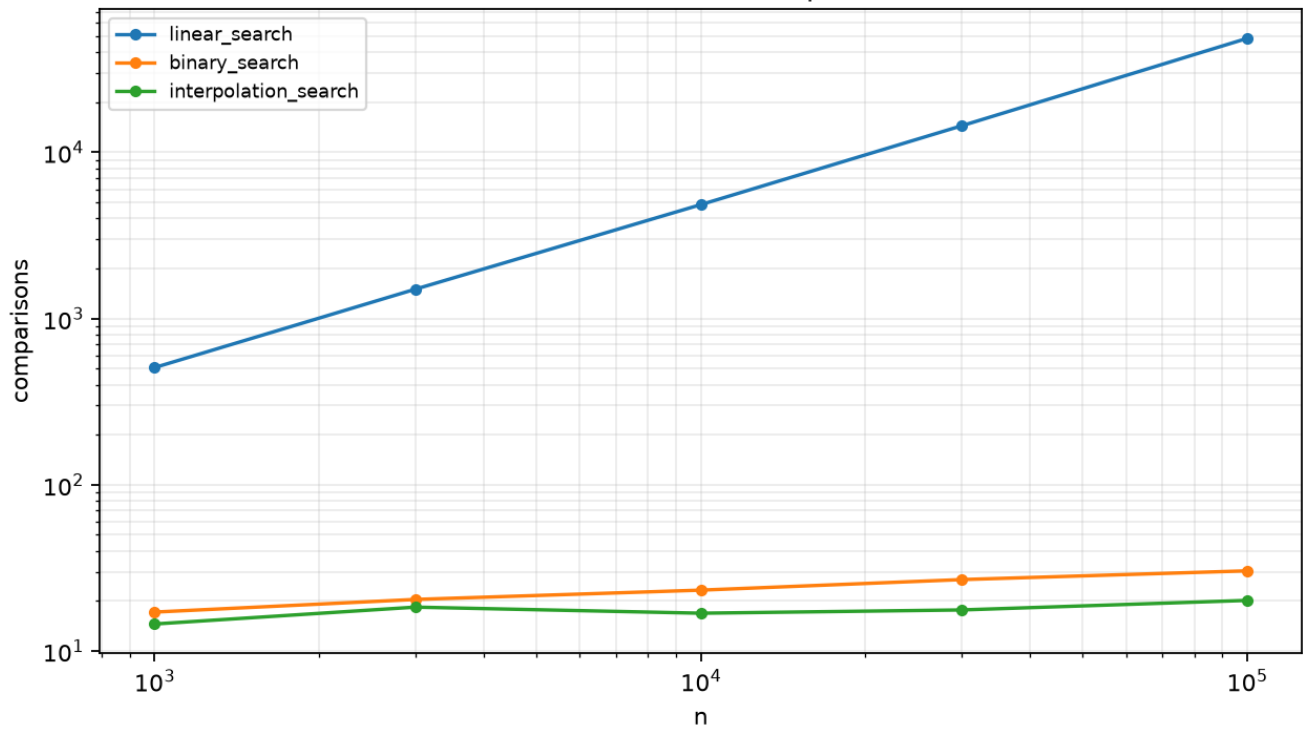
ЛР 6: exponential, miss, comparisons



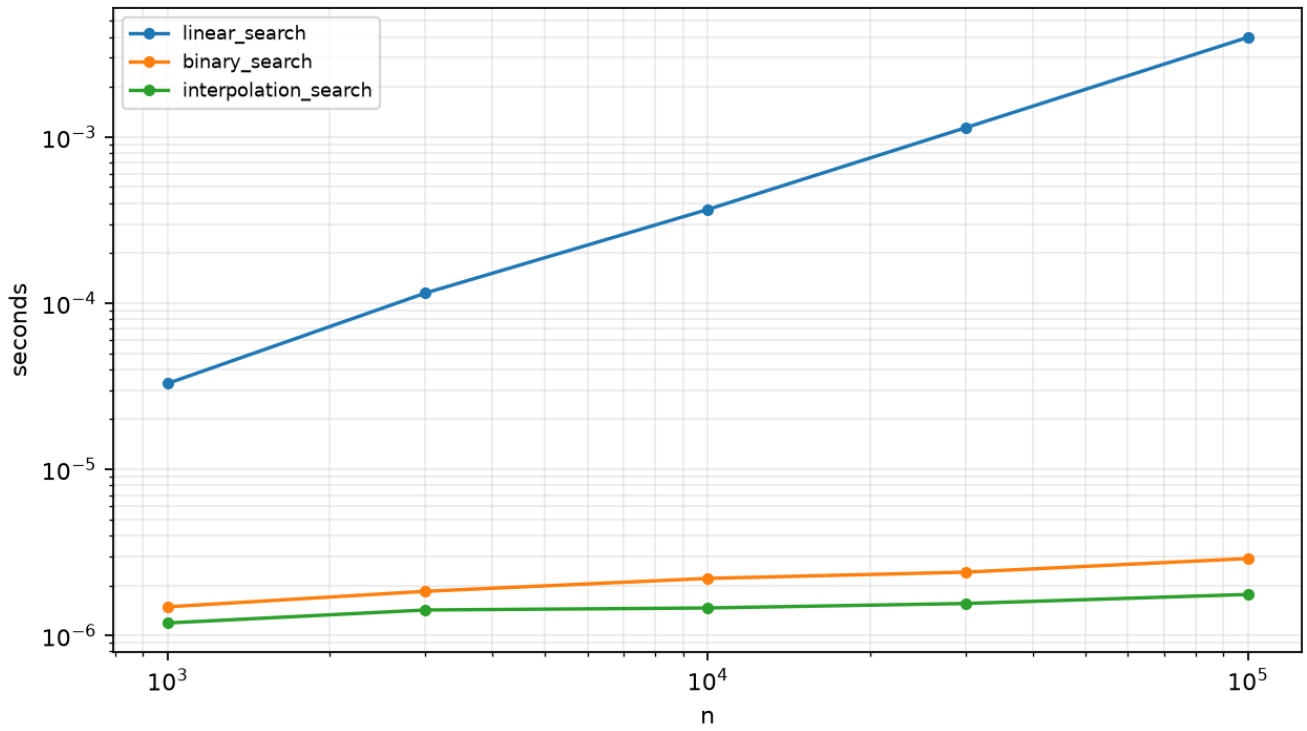
ЛР 6: exponential, miss, seconds



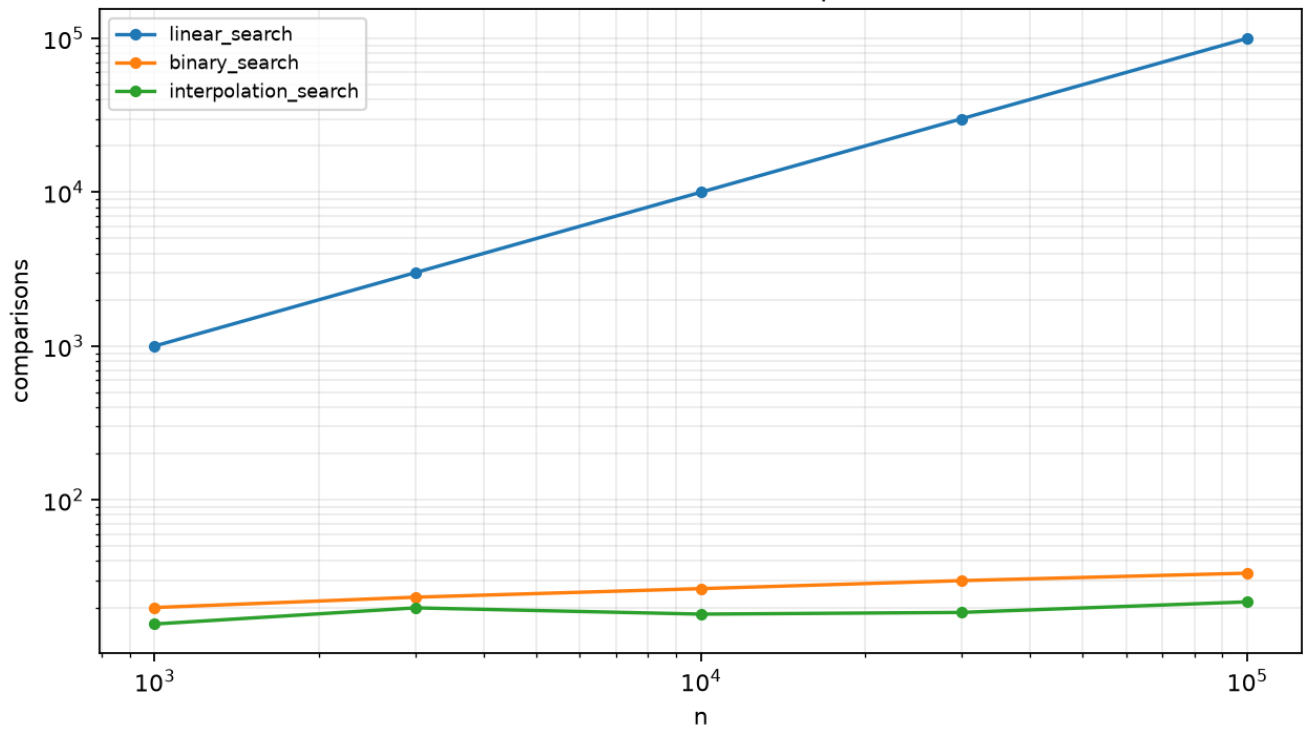
ЛР 6: uniform, hit, comparisons

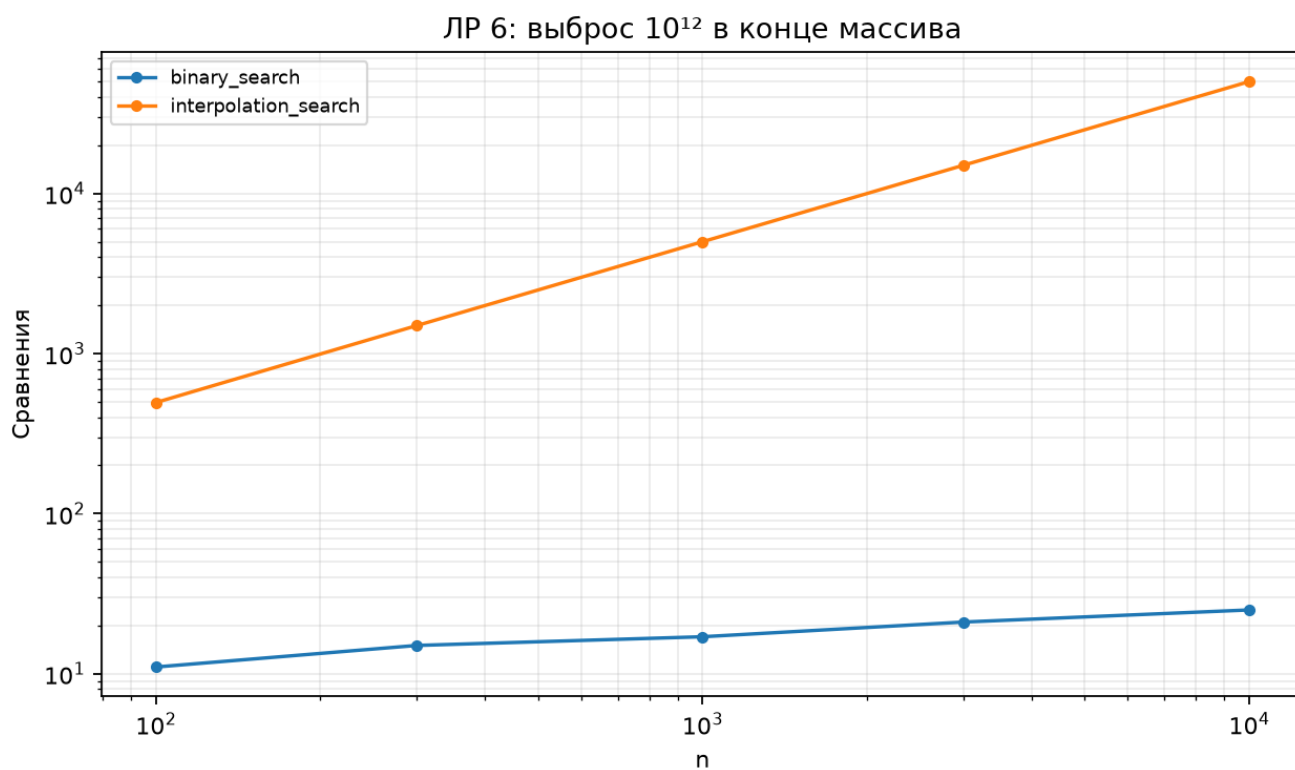
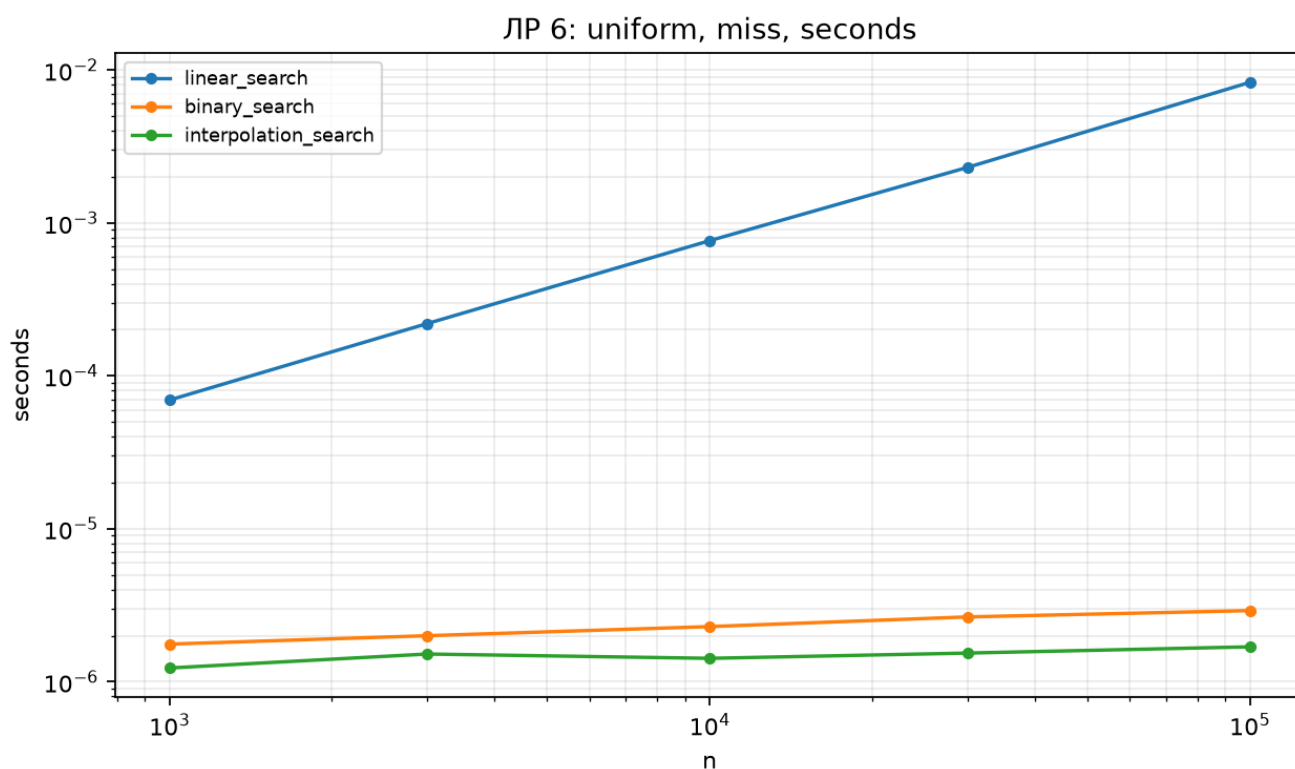


ЛР 6: uniform, hit, seconds



ЛР 6: uniform, miss, comparisons





Исходные данные отчёта

CSV содержит медиану и все пять длительностей trial_1...trial_5. Вспомогательные таблицы без времени содержат точные счётчики или свойства структуры.

- inputs.csv
- measurements.csv

- worst_case.csv
- Паспорт окружения и контрольные суммы